

SCHOOL OF COMPUTER SCIENCE

UNIVERSITY OF PETROLEUM & ENERGY STUDIES, DEHRADUN - 248007, UTTARAKHAND

MAJOR PROJECT MID-TERM DEFENSE QUESTION BANK

Comprehensive Viva Preparation & Technical Defense Guide (50 Questions & Model Answers)

Project: *Autonomous Research Report Agent: An Evidence-Driven Multi-Agent Framework for Automated Research*

Specialization / Batch	SAP ID	Roll No / Reg ID	Name
AIML, Batch 8	500119031	R2142230351	Devansh Saini
AIML, Batch 8	500119592	R2142230632	Yash Thakur
AIML, Batch 7	500121743	R2142230345	Kanishq Vikram Singh
AIML, Batch 7	500119644	R2142230187	Sumit Kumar

Project Mentor: Dr. Anish Kumar Vishwakarma (Assistant Professor, School of Computer Science)
Department of Informatics | Academic Session: 2025–2026

How to Use This Question Bank:

This question bank is engineered specifically for your Major Project Mid-Term Evaluation and Viva Defense. It contains **50 targeted questions** categorized into 9 technical dimensions—from algorithmic mathematical formulations (BM25, Porter stemmer, dense cosine ranking) to experimental benchmarks, multi-agent schema contracts, and aggressive examiner defense ('Grill Me' questions). Each entry provides an in-depth **Model Answer** alongside a **High-Yield Viva Tip** for quick 10-second recall under examination pressure.

Category 1: Project Overview, Motivation & Novelty

Q1. What is the core problem that your Autonomous Research Report Agent (ARA) is solving?

Model Answer: Conventional LLMs and standard RAG systems suffer from four major vulnerabilities when applied to academic research: (1) Superficial context stuffing where either short 100-word web snippets or entire 15-page PDFs are shoved into prompts, causing severe 'Lost-in-the-Middle' context decay; (2) Confirmation bias where naive search queries only retrieve confirmatory evidence; (3) Severe API bottlenecks and HTTP 429 rate limits caused by calculating dense vector embeddings across hundreds of uncured candidate documents; and (4) Citation hallucination. ARA solves these through formal planning contracts, tri-source academic discovery, two-stage cascade indexing, and passage-level evidence grounding.

■ **Viva Tip / Key Recall:** *Keep it crisp: mention 'Lost-in-the-Middle', 'Confirmation Bias', 'API Bottlenecks', and 'Citation Hallucination'.*

Q2. Why can't a researcher simply use ChatGPT, Perplexity, or standard Google Scholar search?

Model Answer: Standard ChatGPT has a static knowledge cutoff and hallucinates citations. Perplexity and web search agents rely almost exclusively on commercial web search APIs (Bing/Google), completely ignoring structured academic citation graphs (OpenAlex, Semantic Scholar) and peer-reviewed metadata. Furthermore, they do not formulate competing hypotheses or check falsification criteria, nor do they provide character-level auditability back to exact paragraphs within arXiv PDFs. Google Scholar provides records but cannot autonomously read, chunk, re-rank, and synthesize findings.

■ **Viva Tip / Key Recall:** *Emphasize that Perplexity is a web search summarizer, whereas ARA is an evidence-grounded scientific research framework.*

Q3. In one minute, what is the core technical novelty of your project?

Model Answer: Our novelty consists of three architectural innovations: (1) A Two-Stage Cascade Hybrid Retrieval Engine combining pure-Python local BM25 scoring with Gemini dense vector re-ranking, cutting embedding API calls by 93% and running on the CPU in 21 milliseconds; (2) A Sliding-Window Scientific Passage Chunker (350 words, 50-word overlap) that extracts the top 5 high-density empirical paragraphs from 15-page PDFs to prevent transformer attention decay; and (3) A Hierarchical Academic Deduplication Engine resolving preprints and published papers across DOI, arXiv ID, URL, and fuzzy title matching with 92.2% efficiency.

■ **Viva Tip / Key Recall:** *Memorize the numbers: '93% API reduction', '21 ms local CPU BM25', '350w/50w overlap', '92.2% deduplication efficiency'.*

Q4. Why did you choose an evidence-driven multi-agent framework rather than a single prompt-chained script?

Model Answer: Prompt chaining tightly couples reasoning, retrieval, and synthesis into a monolithic black box where failure in one stage corrupts the entire output. A multi-agent framework decouples responsibilities across distinct agents (Planner, Researcher, Validator, Synthesizer) governed by formal schema contracts (such as data/research_plan.json). This allows us to independently benchmark, unit-test, and optimize each stage—for example, evaluating the retrieval engine's precision without invoking LLM generation costs.

■ **Viva Tip / Key Recall:** *Explain that agents enforce formal schema contracts rather than conversational back-and-forth.*

Q5. What are the primary application domains where ARA provides the highest value?

Model Answer: ARA is designed for: (1) Systematic scientific literature reviews across computer science, biomedicine, and engineering; (2) Evidence-based prior art discovery for algorithmic and patent research; (3) Technical due diligence comparing conflicting engineering benchmarks; and (4) Academic research assistance helping graduate and doctoral scholars formulate and test hypotheses with counter-evidence.

■ **Viva Tip / Key Recall:** *Relate this directly to Section 1.4 of your report.*

Category 2: System Architecture & Multi-Agent Planning (Phase 1)

Q6. Explain the end-to-end architecture of your system across its 4 phases.

Model Answer: Phase 1 (Completed): Research Planner decomposes user inquiries into domain categories, 3-5 sub-questions, working and competing hypotheses, and falsification criteria, outputting data/research_plan.json. Phase 2 (Completed): Tri-source discovery across Tavily, OpenAlex, and Semantic Scholar, hierarchical deduplication, 6-step semantic indexing, two-stage BM25/Vector filtering, and sliding-window PDF passage chunking, emitting data/research_evidence.json. Phase 3 (Planned for End-Term): Sentence-level Natural Language Inference (NLI) evidence validation and autonomous iterative re-search loops. Phase 4 (Planned for End-Term): Calibrated epistemic report synthesis and direct headless PDF compilation.

■ **Viva Tip / Key Recall:** *Clearly delineate Mid-Term deliverables (Phases 1 & 2) from End-Term deliverables (Phases 3 & 4).*

Q7. What is the role of the Research Planner and why is domain classification necessary?

Model Answer: The Planner (planner/planner.py) prevents undirected search. When given an open inquiry, it performs domain classification (Technical, Biomedical, Legal, General). This classification is necessary because retrieval parameters must adapt: technical papers require arXiv preprint queries and benchmark metrics, biomedical inquiries prioritize peer-reviewed DOI indexed works with clinical trial consensus, and legal inquiries prioritize statutory authorities.

■ **Viva Tip / Key Recall:** *Highlight that different domains have fundamentally different evidence standards.*

Q8. How does your system address confirmation bias during the planning stage?

Model Answer: Standard search agents formulate queries that only look for confirmatory evidence. In ARA, the Planner is strictly prompted to generate both a primary Working Hypothesis and Competing Alternative Hypotheses, along with explicit empirical Falsification Criteria. These competing hypotheses generate targeted sub-queries that force the search agent to discover dissenting views, benchmark degradations, and counter-evidence.

■ **Viva Tip / Key Recall:** *Mention Karl Popper's falsification criterion: science requires actively attempting to disprove the hypothesis.*

Q9. What is the exact data contract produced by Phase 1, and where is it stored?

Model Answer: Phase 1 serializes a machine-readable JSON artifact at data/research_plan.json. It contains: domain classification, query list, orthogonal sub-questions (addressing foundations, mechanisms, benchmarks, and trade-offs), working hypothesis, competing hypotheses, falsification criteria, and required evidence quotas (minimum peer-reviewed sources, quantitative tables, and counter-evidence count).

■ **Viva Tip / Key Recall:** *Mention that this contract guarantees deterministic, testable boundaries for Phase 2.*

Q10. How does the system ensure that the LLM produces valid JSON schemas without formatting errors?

Model Answer: We enforce strict JSON output schemas via structured prompting, low-temperature sampling (0.1–0.2), and defensive validation parsing with fallbacks. If an LLM response contains markdown wrapping (e.g. ``json ... ``), regex sanitizers extract the inner JSON string before schema verification.

■ **Viva Tip / Key Recall:** *Explain that robustness against non-deterministic completions is a core engineering requirement.*

Category 3: Academic Discovery & Multi-Key Deduplication (Phase 2A & 2B)**Q11. Why did you integrate three search providers (Tavily, OpenAlex, Semantic Scholar) instead of just one?**

Model Answer: Each provider has unique strengths: Tavily accesses the live web for contemporary whitepapers, engineering blogs, and newest preprints; OpenAlex accesses over 250 million scholarly records with rich metadata, open-access full-text PDF links, and institutional affiliations; Semantic Scholar provides academic citation graphs and influential citation metrics. Combining them provides a comprehensive, multi-angle scientific corpus.

■ **Viva Tip / Key Recall:** *Call it 'Tri-Provider Scholarly Ingestion'.*

Q12. How does the system handle unauthenticated API rate limits on Semantic Scholar or OpenAlex?

Model Answer: Our unified asynchronous discovery layer (`search_agent/search_engine.py`) implements exponential backoff with jitter and graceful degradation. If Semantic Scholar returns HTTP 429 (Too Many Requests), the engine logs the event, automatically falls back to OpenAlex and Tavily academic search routes, and continues execution without terminating the pipeline.

■ **Viva Tip / Key Recall:** *Evaluators love fault tolerance. Mention 'exponential backoff' and 'graceful fallback'.*

Q13. How does your Hierarchical Multi-Key Deduplication Engine work?

Model Answer: In `search_agent/sources.py`, deduplication operates in a 4-level hierarchy: (1) DOI Matching: If two records share a Digital Object Identifier, they are merged; (2) arXiv Identifier Matching: Canonical arXiv IDs (e.g., 2401.09136) are normalized and matched; (3) Normalized URL Matching: Query parameters, trailing slashes, and protocol variations are stripped; (4) Fuzzy Title Matching: Levenshtein distance similarity (>90%) with normalized punctuation consolidates preprints with their final journal titles.

■ **Viva Tip / Key Recall:** *Mention the hierarchy: DOI -> arXiv ID -> URL -> Fuzzy Title.*

Q14. What were your empirical deduplication results in experimental benchmarks?

Model Answer: Across benchmark executions on complex scientific prompts, the discovery engine harvested 539 gross candidate sources. Our multi-key deduplication module merged 42 redundant records, retaining 497 unique academic sources, achieving a 92.2% deduplication retention efficiency while eliminating duplicate citations in the bibliography.

■ **Viva Tip / Key Recall:** *Quote Section 6.1 of the report directly.*

Q15. What is Logarithmic Citation Authority Scaling and why did you implement it?

Model Answer: If raw citation counts are used directly for ranking, foundational papers from 2017 with 50,000 citations (like 'Attention Is All You Need') would completely dominate every search ranking, drowning out critical 2025 breakthroughs that have zero or few citations simply due to recency. We apply a logarithmic dampening function: $\text{Authority} = \log(1 + \text{citations}) / \log(1 + \text{max_citations})$. This normalizes credibility without penalizing recent preprints.

■ **Viva Tip / Key Recall:** *Formula: $\text{Authority} = \log(1 + \text{citations}) / \log(1 + \text{max_citations})$. This shows strong mathematical maturity.*

Category 4: The 6-Step IR Engine & Mathematical Foundations

Q16. Walk the panel through the 6 steps of your Information Retrieval engine.

Model Answer: Step 1: Stop-Word Removal using a tailored academic stop-word lexicon. Step 2: Pure-Python Porter Stemming implementing all 5 stages of morphological suffix stripping (conforming strictly to Porter, 1980). Step 3: Inverted Index & BM25 Scoring to compute probabilistic lexical relevance. Step 4: Two-Stage Coarse Filtering narrowing ~500 candidates down to 35. Step 5: Dense Vector Embedding via Google Gemini text-embedding-004 and Cosine Similarity calculation. Step 6: Prompt-Aligned Hybrid Re-ranking combining BM25 and vector scores.

■ **Viva Tip / Key Recall:** *Walk through Step 1 to Step 6 sequentially as implemented in `search_agent/semantic_indexer.py`.*

Q17. Explain the mathematical formulation of Robertson–Spärck Jones BM25 term weighting.

Model Answer: BM25 scores a query Q against document D using: $\text{Score}(D, Q) = \sum_{q \in Q} [\text{IDF}(q) * (f(q, D) * (k_1 + 1)) / (f(q, D) + k_1 * (1 - b + b * (|D| / \text{avgl})))]$. Here, $f(q, D)$ is term frequency, $|D|$ is document length, avgl is average document length across the corpus, $k_1 = 1.5$ governs term frequency saturation, and $b = 0.75$ controls the degree of document length penalization.

■ **Viva Tip / Key Recall:** *Mention parameters $k_1 = 1.5$ and $b = 0.75$. Explain that k_1 prevents a term repeated 20 times from having 20x the weight.*

Q18. What is Inverse Document Frequency (IDF) in BM25 and why does it matter?

Model Answer: IDF penalizes common terms across the entire corpus and rewards rare, high-signal terms. In BM25, $IDF(q) = \ln((N - n(q) + 0.5) / (n(q) + 0.5) + 1)$, where N is the total number of documents and $n(q)$ is the number of documents containing query term q . A term like 'retrieval' appearing in almost every document gets low IDF, whereas a specific term like 'falsification' gets high IDF.

■ **Viva Tip / Key Recall:** Explain that IDF ensures informative domain keywords dominate the score.

Q19. Why did you write a custom Porter Stemmer in pure Python instead of importing NLTK or spaCy?

Model Answer: NLTK and spaCy are heavy external dependencies with compiled C-extensions, large data package downloads (nltk.download('punkt')), and platform-specific compilation overhead. By engineering a self-contained, 5-stage Porter stemmer in pure Python, our system runs with 100% portability across Windows, Linux, and minimal Docker containers with zero binary dependencies and instant startup time.

■ **Viva Tip / Key Recall:** Evaluators value dependency hygiene and lightweight pure-Python engineering.

Q20. How is Cosine Similarity computed in Step 5?

Model Answer: Cosine similarity measures the cosine of the angle between two non-zero vectors in multi-dimensional semantic space: $\text{CosSim}(u, v) = (u \cdot v) / (||u|| * ||v||)$. In our pipeline, u is the 768-dimensional dense embedding of the research sub-question, and v is the candidate's embedding from Google Gemini text-embedding-004. Values range from -1.0 to +1.0 (typically 0.0 to 1.0 for normalized text embeddings).

■ **Viva Tip / Key Recall:** Explain that dot product over magnitude product captures semantic orientation regardless of length.

Q21. How do you combine BM25 scores and Vector Cosine scores in Step 6?

Model Answer: Because BM25 yields unbounded positive scores while Cosine Similarity is bounded between 0 and 1, we first min-max normalize BM25 scores across the candidate set into the range [0.0, 1.0]. We then calculate the final hybrid score as: $\text{FinalScore} = 0.35 * \text{Normalized_BM25} + 0.65 * \text{Dense_Cosine_Similarity}$. This balances exact keyword precision (35%) with semantic conceptual match (65%).

■ **Viva Tip / Key Recall:** Explain min-max normalization: $(\text{score} - \text{min}) / (\text{max} - \text{min})$, followed by the 35/65 weighted sum.

Category 5: Two-Stage Hybrid Filtering & Optimization

Q22. What is the two-stage candidate filtering mechanism and why is it essential?

Model Answer: In a naive RAG system, discovering 500 candidate papers across 3 APIs means making 500 remote embedding API requests. This creates 10+ seconds of network latency, costs money, and frequently crashes due to HTTP 429 rate limits. In our two-stage cascade: Stage 1 uses localized, in-memory BM25 to coarse-filter the 500 candidates down to 35 in 21 milliseconds on the CPU. Stage 2 generates dense embeddings solely for those 35 high-probability candidates.

■ **Viva Tip / Key Recall:** This is one of your strongest benchmark results: 500 -> 35 on CPU in 0.021s.

Q23. What are the exact performance numbers for your two-stage optimization?

Model Answer: In our experimental benchmarks on 497 unique candidates: Stage 1 BM25 coarse filtering executed locally on the CPU in exactly 0.021 seconds (21 milliseconds), reducing candidates from 497 to 35. Stage 2 dense re-ranking required only 35 embedding calls instead of 497, achieving an exact 93.0% reduction in vector embedding API consumption.

■ **Viva Tip / Key Recall:** State clearly: '0.021 seconds latency' and '93.0% API reduction'.

Q24. Does filtering 500 candidates down to 35 risk discarding semantically relevant papers (False Negatives)?

Model Answer: We conducted recall validation tests. Because scientific papers feature highly informative technical keywords in their titles and abstracts, the top 35 BM25 pool captures over 96% of semantically relevant works. By setting the cutoff at 35 (rather than 10 or 15), we maintain high candidate recall while keeping the downstream dense re-ranking batch well within API rate boundaries.

■ **Viva Tip / Key Recall:** Explain that technical literature is lexically rich, making BM25 an ideal coarse filter.

Q25. How do you handle cases where a query has synonyms not present in the paper title?

Model Answer: That is precisely why Stage 2 dense vector embedding is paired with Stage 1. Even if a paper only matches 1 or 2 keywords in BM25, its inclusion in the top 35 allows Stage 2 dense embeddings to recognize semantic equivalence (e.g. 'hallucination reduction' vs 'factual grounding'). Furthermore, Phase 1 Planner generates multiple orthogonal sub-queries with diverse terminology.

■ **Viva Tip / Key Recall:** *Explain that the Planner's multi-angle queries provide vocabulary diversification.*

Q26. What is the memory footprint and CPU overhead of running BM25 locally?

Model Answer: The memory footprint is negligible: an in-memory inverted index of 500 titles and abstracts occupies less than 2.5 megabytes of RAM. The scoring involves simple integer frequency lookups and floating-point arithmetic, which is why execution finishes in 21 milliseconds on a single standard CPU thread.

■ **Viva Tip / Key Recall:** *Highlight that no external vector database daemon (Pinecone, Chroma, Milvus) is required for Phase 2 indexing.*

Category 6: Scientific PDF Processing & 'Lost-in-the-Middle' (Phase 2C)

Q27. What is the 'Lost-in-the-Middle' phenomenon and how does it affect scientific RAG?

Model Answer: Documented by Liu et al. (2024) in Transactions of the ACL, 'Lost in the Middle' proves that language models retrieve information accurately from the beginning and end of long input contexts, but their attention degrades precipitously (up to 40% performance drop) when relevant evidence is positioned within the middle 40%–60% of the prompt. Dumping entire 15-page PDFs into prompts causes the LLM to overlook critical quantitative metrics and ablation tables buried in the middle.

■ **Viva Tip / Key Recall:** *Cite the paper: Liu et al. (2024), Transactions of the Association for Computational Linguistics.*

Q28. How does your Deep PDF Passage Chunking algorithm solve this?

Model Answer: When an arXiv scientific PDF is parsed via pypdf (averaging 10,000 to 15,000 words), documents $\geq 3,500$ characters are segmented into 350-word sliding windows with a 50-word overlap. Each passage chunk is scored against the research sub-question using our hybrid indexing algorithm. Only the top 5 highest-signal empirical passages are extracted and passed to the LLM prompt.

■ **Viva Tip / Key Recall:** *Parameters: 350-word window, 50-word overlap, top 5 passages extracted.*

Q29. Why did you choose a 350-word window with a 50-word overlap instead of standard 1000-token chunks?

Model Answer: A 350-word chunk corresponds roughly to 1 to 2 dense scientific paragraphs—the exact semantic unit where an author presents an experimental hypothesis, methodology, or quantitative result. The 50-word sliding overlap ensures that sentences crossing chunk boundaries are not truncated mid-premise. Larger 1000-token chunks re-introduce context dilution, while smaller 100-word chunks sever empirical context.

■ **Viva Tip / Key Recall:** *Explain that 350 words is the natural semantic boundary of a scientific paragraph.*

Q30. How does ARA download and parse arXiv PDFs in real time?

Model Answer: In search_agent/content_retriever.py, when a discovered candidate contains an arXiv ID or open-access PDF link, the engine downloads the PDF stream with a strict timeout and user-agent header. It parses text page-by-page using pypdf, removes non-printable ligature artifacts and header/footer boilerplates, and passes the clean text stream to the passage chunker.

■ **Viva Tip / Key Recall:** *Mention handling of timeouts and text sanitization.*

Q31. How much prompt token reduction does passage chunking achieve?

Model Answer: A standard 15-page scientific preprint averages 10,000 to 15,000 words (~13,000 to 20,000 tokens). Extracting the top 5 passages of 350 words totals approximately 1,750 words (~2,300 tokens). This achieves over a 70% to 85% reduction in token overhead per paper, eliminating prompt token bloat while ensuring high attention focus.

■ **Viva Tip / Key Recall:** *Numbers: from 15,000 words down to 1,750 words = >70% token reduction.*

Category 7: Software Engineering, Testing & Verification

Q32. How many automated unit tests validate your repository, and what is your pass rate?

Model Answer: A total of 170 unit tests pass cleanly with 100% regression stability using pytest. These span: tests/test_semantic_indexer.py (7 tests covering stemmer invariance, BM25 weighting, cosine ranking, and chunking), tests/test_phase2_completion.py (18 tests covering connectors and fallback logic), tests/test_cli_ui.py (18 tests covering terminal progress rendering), and tests/test_end_to_end_research_eval.py (28 tests verifying complete contract schema compliance).

■ **Viva Tip / Key Recall:** State clearly: '170 unit tests, 100% passing cleanly under pytest'.

Q33. What is the purpose of demo_semantic_pipeline.py?

Model Answer: demo_semantic_pipeline.py is a standalone, offline interactive verification script built with rich. It allows examiners to visualize all 7 stages of indexing in real time: raw text tokenization, stop-word removal, 5-stage Porter stemming roots, in-memory BM25 inverted index tables, dense vector generation, cosine ranking scores, and sliding-window PDF passage chunking on a sample scientific document without incurring live API costs.

■ **Viva Tip / Key Recall:** Offer to run demo_semantic_pipeline.py live on the terminal during the viva!

Q34. How is the codebase structured across modules and directories?

Model Answer: The repository adheres to modular separation: planner/ handles Phase 1 decomposition and hypothesis contracts; search_agent/ contains search_engine.py (connectors), sources.py (deduplication), semantic_indexer.py (BM25, embeddings, chunker), content_retriever.py (PDF parsing), and evidence_extractor.py (fact extraction); reports/ contains HTML and document generators; and tests/ contains the 170-test validation suite.

■ **Viva Tip / Key Recall:** Point to Section 5.1 Table in the Mid-Term Report.

Q35. How do you prevent environment drift and ensure reproducible builds across machines?

Model Answer: All dependencies are pinned in requirements.txt and isolated within Python virtual environments. Because our IR engine (Porter stemmer, BM25, and deduplication) is engineered in pure Python with zero compiled C-extensions, the repository builds and passes all 170 tests identically across Windows, macOS, and Linux without native build-tool dependencies.

■ **Viva Tip / Key Recall:** Emphasize 'pure-Python portability' and 'pinned requirements.txt'.

Q36. How do you monitor and display execution progress in the terminal CLI?

Model Answer: In main.py and the ui/ module, we utilize the rich library to display real-time terminal progress spinners, live provider status indicators (Tavily, OpenAlex, Semantic Scholar), harvested candidate counts, deduplication stats, and elapsed timestamps.

■ **Viva Tip / Key Recall:** Mention user experience: academic researchers need visibility into long-running multi-source harvesting.

Category 8: End-Term Roadmap, Phase 3 & 4 Deliverables

Q37. What is planned for Phase 3 (Evidence Validation) in Major Project Part 2?

Model Answer: Phase 3 introduces an automated Evidence Validation Layer. It will implement sentence-level Natural Language Inference (NLI) to audit extracted claims against the raw PDF text, classifying claims as Entailment, Contradiction, or Neutral. It will also compute an Evidence Sufficiency Scorecard checking whether required quantitative data points and counter-evidence quotas have been satisfied.

■ **Viva Tip / Key Recall:** Mention sentence-level NLI: checking if the source text actually entails the extracted claim.

Q38. How will the Autonomous Iterative Re-Search Loop work in Phase 3?

Model Answer: If the Evidence Sufficiency Scorecard flags gaps—such as an absence of quantitative benchmark comparisons or missing counter-evidence for a competing hypothesis—the agent will not proceed to synthesis. Instead, it will autonomously formulate targeted secondary queries, query the search connectors again, and index supplementary literature until evidence sufficiency criteria are fulfilled.

■ **Viva Tip / Key Recall:** *This transforms ARA from a feed-forward pipeline into a true reflective agent.*

Q39. What is Epistemic Confidence Calibration in Phase 4?

Model Answer: Rather than presenting all LLM assertions with unearned certainty, the synthesizer will calculate a calibrated epistemic confidence score (ranging from 0.0 to 1.0) for every major section and conclusion. The score is calibrated against: the number of independent peer-reviewed sources supporting the finding, total citation authority, the absence of empirical contradictions, and the presence of verified quantitative data.

■ **Viva Tip / Key Recall:** *Explain that scientific reports must distinguish between established consensus and speculative preprints.*

Q40. How will deterministic citation anchors work in the final report?

Model Answer: Every claim in the synthesized report will feature deterministic numerical citation anchors ([1], [2]) linking directly to the bibliography. Each bibliographic entry contains the canonical URL (prioritizing DOI, publisher URL, or arXiv landing page), publication year, author list, and the exact verbatim quote extracted from the paper.

■ **Viva Tip / Key Recall:** *Point out that this eliminates citation fabrication entirely.*

Q41. How will the final report be compiled into a publication-ready PDF in Phase 4?

Model Answer: We are integrating WeasyPrint to directly compile publication-grade, paginated PDF documents from the final HTML report artifact. WeasyPrint supports CSS3 paged media specifications (@page rules, page-break controls, running headers, and footnote anchors), producing clean print deliverables headlessly.

■ **Viva Tip / Key Recall:** *Mention WeasyPrint and CSS3 Paged Media.*

Category 9: Rapid-Fire & Tough Defense Questions ('Grill Me' Section)**Q42. 'Isn't your project just a wrapper around Google Gemini and Tavily?' (Tough Examiner Question)**

Model Answer: No, absolutely not. The LLM (Gemini) is only invoked for reasoning tasks: decomposing the initial topic in Phase 1 and embedding the top 35 candidates in Phase 2. The entire retrieval backbone—tokenization, stop-word elimination, 5-stage Porter stemming, in-memory BM25 inverted indexing, two-stage cascading filters, multi-key deduplication across DOIs and arXiv IDs, and sliding-window PDF passage chunking—is engineered custom in pure Python. Even if we swapped Gemini for an open-source model like Llama 3, the entire retrieval architecture remains 100% operational.

■ **Viva Tip / Key Recall:** *Be confident: explain that the core algorithmic engineering is in the retrieval and chunking pipeline, not the LLM.*

Q43. What happens if all three search APIs fail simultaneously due to a network outage?

Model Answer: The system includes an offline demo harness and cached local fixtures (tested in tests/test_phase2_completion.py). In production, if network requests timeout, the connector raises structured exception classes that trigger retry mechanisms; if unresolvable, the system logs the provider error and gracefully synthesizes a report based on whatever cached or partial evidence was successfully retrieved.

■ **Viva Tip / Key Recall:** *Mention graceful degradation and structured exceptions.*

Q44. What is the computational complexity of your BM25 scoring algorithm?

Model Answer: Building the inverted index takes $O(N * L)$ time, where N is the number of documents (~500) and L is average document length in tokens (~200). Query scoring takes $O(|Q| * D_{avg})$ time, where $|Q|$ is the number of query tokens (typically 4–8) and D_{avg} is the average posting list length. In practice, this requires fewer than 10,000 simple floating-point operations, executing in ~21 milliseconds.

■ **Viva Tip / Key Recall:** *State $O(N * L)$ index time and $O(|Q| * D_{avg})$ query time.*

Q45. Why did you choose BM25 over TF-IDF?

Model Answer: TF-IDF has a linear term frequency term: if a word appears 20 times, it gets 20 times the score, making it vulnerable to keyword stuffing. BM25 solves this with term frequency saturation (controlled by parameter $k_1 = 1.5$), meaning that after 3 or 4 occurrences, additional repetitions yield diminishing returns. Furthermore, BM25 incorporates document length normalization (parameter $b = 0.75$), penalizing long documents that match keywords purely due to verbosity.

■ **Viva Tip / Key Recall:** *Explain term frequency saturation and document length normalization.*

Q46. Can you guarantee that ARA will never hallucinate a fact?

Model Answer: No LLM system can offer a 100% theoretical mathematical guarantee against hallucination. However, ARA mitigates hallucination through three architectural constraints: (1) Deterministic character-level quote extraction where findings must be grounded in raw PDF passage text; (2) Sentence-level NLI entailment audits in Phase 3; and (3) Explicit epistemic confidence scoring, which downgrades unverified claims to 'Inconclusive' rather than presenting them as factual.

■ **Viva Tip / Key Recall:** *Never claim 100% zero hallucination; academic evaluators will reject that. Instead, explain your layered mitigation architecture.*

Q47. Why didn't you use an established vector database like ChromaDB or Pinecone?

Model Answer: ChromaDB and Pinecone are designed for persistent multi-gigabyte vector corpora across millions of documents. In ARA, each research inquiry dynamically harvests an ad-hoc, ephemeral candidate pool of ~500 papers. Spinning up a separate vector database daemon introduces unnecessary operational overhead, disk dependencies, and network roundtrips. In-memory numpy cosine similarity across 35 vectors takes 0.4 milliseconds with zero infrastructure overhead.

■ **Viva Tip / Key Recall:** *This demonstrates senior architectural decision-making: choosing the right tool for the specific problem scale.*

Q48. How do you handle OCR noise and multi-column formatting in downloaded arXiv PDFs?

Model Answer: pypdf extracts text streams across PDF layout text objects. We apply a series of regex sanitization filters: joining hyphenated words broken across line breaks (e.g. 'retrie-val' -> 'retrieval'), stripping repeated academic header/footer strings, and removing non-printable binary font-encoding artifacts before text enters the sliding-window chunker.

■ **Viva Tip / Key Recall:** *Mention de-hyphenation and header/footer stripping.*

Q49. How do you assess the quality of an academic paper beyond raw citation counts?

Model Answer: In addition to logarithmic citation authority scaling, ARA evaluates: (1) Publication venue credibility (distinguishing peer-reviewed conference/journal proceedings indexed in OpenAlex from unreviewed preprints); (2) Recency weighting to prioritize contemporary benchmarks; and (3) Empirical density, measuring whether the full-text paper contains quantitative benchmark tables and experimental sections.

■ **Viva Tip / Key Recall:** *Explain that peer-review status, recency, and quantitative density are combined.*

Q50. If you had to summarize the entire project in one final sentence for the evaluation committee, what would it be?

Model Answer: 'Autonomous Research Report Agent transitions automated scientific synthesis from naive, unverified prompt-chaining to an auditable, two-stage hybrid retrieval framework that plans competing hypotheses, queries heterogeneous scholarly graphs, eliminates attention decay through passage chunking, and guarantees source-grounded research integrity.'

■ **Viva Tip / Key Recall:** *Deliver this closing sentence with confidence!*